

Trading Simulator

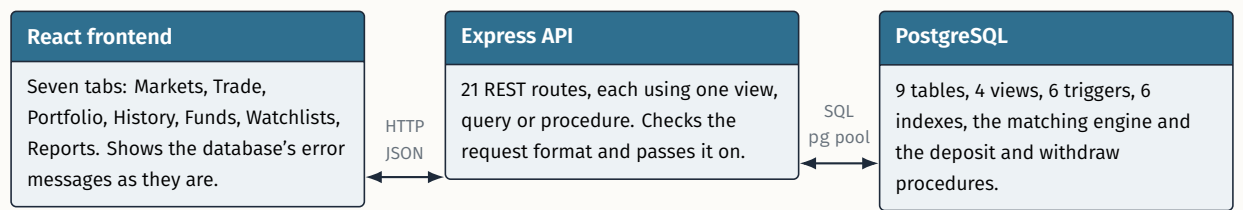
Application design, technology stack and progress

Group 19	Prateek Thota 2024AAPS0254H	Aditya Sanadhaya 2024AAPS0263H	Shreyas Khunteta 2024AAPS0302H	Muhilan Selvakumar 2024AAPS0309H	11 October 2026
----------	---------------------------------------	--	--	--	-----------------

7 WORKFLOWS	21 API ROUTES	9 TABLES	4 VIEWS	6 TRIGGERS	6 INDEXES
----------------	------------------	-------------	------------	---------------	--------------

1 Architecture

The app has three layers: a React frontend, an Express API and a PostgreSQL database. All the main logic (matching orders, moving cash and shares, business rules) is in the database. The backend and frontend mostly pass data through.



2 Technology stack

LAYER	TECHNOLOGY	WHY WE CHOSE IT
Database	PostgreSQL, hosted on Neon	Has stored procedures, triggers, window functions, partial indexes and advisory locks, which the matching engine needs
Backend	Node.js, Express, the pg driver	Plain SQL, no ORM, so every query the app runs is easy to see and explain
Frontend	React with Vite	One component per screen, and a small hook for loading and errors

3 Workflows

The seven workflows the app supports:

WORKFLOW	SCREEN AND ROUTE	DATABASE OBJECTS	WHAT IT SHOWS
1 Place an order	Trade POST /api/orders	place_order, compute_fill_plan, trade trigger	Changes five tables (orders, trades, accounts, holdings, cash ledger) in one transaction, with locking
2 Cancel an order	Trade, your orders POST .../:id/cancel	cancel_order	Status change and cash release happen together or not at all
3 Rejected requests	Trade, Funds, Watchlists	checks in the procedures, CHECK, UNIQUE, key constraints, append-only triggers	The database refuses invalid actions and the user sees its message
4 Search the market	Markets GET /api/instruments?search=§or=	ILIKE search, v_order_book, aggregation over trades	Filtering in SQL, best bid and ask from a view
5 Portfolio, history and reports	Portfolio, History, Reports GET .../summary, .../trades, /api/reports/:name	v_account_summary on v_portfolio; queries 2, 3, 5, 6 run straight from queries.sql	A view built on a view, joins, a correlated subquery, a CTE with RANK(), NOT EXISTS
6 Deposit and withdraw	Funds POST .../deposit, .../withdraw	deposit_cash, withdraw_cash, cash_ledger	Balance and ledger row change together; reserved cash can't be withdrawn
7 Watchlists	Watchlists GET, POST, DELETE	watchlists, watchlist_items	UNIQUE (user_id, name) and the composite key stop duplicates; ON DELETE CASCADE

Example: placing a buy order. The user picks an instrument, enters a price and quantity and clicks Buy. The API checks that the ids and quantity are whole numbers and calls `place_order`. In one transaction, the procedure locks the instrument and the accounts involved, checks and reserves the cash, inserts the order, gets the matching sells from `compute_fill_plan`, and for each one writes a trade, moves cash and shares and adds two ledger rows. The trade trigger updates the last price. If anything fails, the whole order is rolled back and the user sees the database's message, for example insufficient available cash: need 1000.00, have 640.00.

4 Validation, errors and concurrency

The browser blocks obviously wrong input. The API only checks the format of a request (ids are whole numbers, amounts are numbers, side is BUY or SELL). The database checks all the business rules: positive prices, enough cash or shares, no duplicate watchlist entries, accounts and instruments that exist. The API doesn't check for duplicates itself. It lets the database constraint reject them and shows the error.

Errors from a procedure or constraint are returned as HTTP 400 with the database's message. An unknown account, instrument, order, watchlist or report in the URL gives 404. Anything else is a 500, which

is logged on the server and not shown to the user.

Two users can place orders at the same time. `place_order` and `cancel_order` first take an advisory lock on the instrument, so orders for one instrument are matched one at a time. Then they lock the accounts involved in increasing id order, so two transactions can't deadlock. `deposit_cash` and `withdraw_cash` lock the account they change, so a withdrawal and an order on the same account don't overlap.

5 Progress

Done. Everything described in this document: the database (tables, seed data, views, triggers, indexes, the six queries, the matching engine and the funds procedures), the backend API and the frontend with all seven workflows.

Left for the final. More testing, and any changes from the mid-sem review.

Not included. Market orders, stop orders and login.

6 Team

MEMBER	PART	COVERS
Prateek Thota	Matching engine and transactions	<code>compute_fill_plan</code> , <code>place_order</code> , <code>cancel_order</code> , the funds procedures; locking and concurrency
Muhilan Selvakumar	Database design and schema	EER diagram, tables, keys and constraints, normalization
Aditya Sanadhaya	Queries, views and indexes	The six retrieval queries, the four views, the six indexes
Shreyas Khunteta	Application and integration	Express API, React frontend, triggers, seed data, the demo workflows